E4 – Orchestrator & End-to-End Sessions

E4-Requirements (Final)

We implement **concrete, testable workflows** that chain E1 (data), E2 (quant), and E3 (LLM tools) into end-to-end "sessions" with clear contracts and no hidden dependencies.

Required sessions:

1. **ThesisEvaluationSession**
   - Input: thesis_id: str (path param) or Thesis object.
   - Steps:
     1. Load thesis by ID via DataAccess.
     2. Run ThesisEvaluationService.evaluate_thesis(thesis) → ThesisEvaluationResult.
     3. Run llm_review_thesis(thesis, eval_result) → ThesisReview.
     4. (Optional but supported hook) Generate a non-binding TradePlan proposal via E5 adapter.
     5. Persist the evaluation result (quant + review) as a single atomic record.
     6. Return a structured ThesisEvaluationSessionResult.
2. **DailyUpdateSession**
   - Input: (optionally) as_of: date, otherwise "today".
   - Steps:
     1. Load **active** theses and current portfolio state via DataAccess.
     2. Compute portfolio "depth" / risk snapshot (via existing get_portfolio_depth or equivalent).
     3. Load recent observations and detect alerts (e.g. disconfirmers, actionable flags, thesis_refs).
     4. Build DailyContext (portfolio change, risk highlights, alerts, observations).
     5. Run llm_daily_summary(context) → DailySummary.
     6. Persist alerts and daily summary.
     7. Return a structured DailyUpdateSessionResult.
3. (Optional, low-priority but coherent) **CrossThesisAnalysisSession / IntuitionQASession**
   - Cross-thesis: Given a list of thesis IDs, fetch them and call llm_cross_theses → CrossThesisReport.

    - IntuitionQA: Given a question and (optional) thesis filter, retrieve observations and call llm_query_intuition.

API endpoints:

- POST /api/v1/thesis/{thesis_id}/evaluate → ThesisEvaluationSessionResult.
- POST /api/v1/session/daily-update → DailyUpdateSessionResult.
- Optional:
    - POST /api/v1/thesis/compare → CrossThesisReport.
    - POST /api/v1/intel/qa → IntuitionAnswer.

Error and persistence rules:

- Thesis evaluation:
    - 404 if thesis not found.
    - If E2 or E3 fails → **no DB writes, 500**. No partial persistence.
- Daily update:
    - If DB/data access fails → 500.
    - If LLM summary fails:
        - We **do not fail the whole session**.
        - We persist alerts and return a synthetic DailySummary with insufficient_context=True and empty lists.
        - HTTP 200 with an explicit "LLM summary failed" note only in logs/metrics, not in schema (schema remains DailySummary).

No session depends on "future magic": all required services are explicit dependencies; there is no assumed risk engine or scheduler beyond what's defined.

E4-Research Summary (Compressed)

- We treat each session as a **local saga / workflow** but in a **single process / single service** setting; no distributed saga engine. The "transaction" boundary is the session.
- For **ThesisEvaluationSession**, we choose **all-or-nothing** persistence: if the pipeline can't produce a valid combination of E2 + E3 outputs, we do not store anything and return a 500. That keeps evaluation logs clean and avoids half-baked evaluation rows.
- For **DailyUpdateSession**, P&L, alerts, and state updates are more important than the LLM narrative. We therefore **degrade gracefully**: LLM failure results in a

DailySummary with insufficient_context=True but does not block alerts/P&L persistence.

- Testing follows the **testing pyramid**: E1–E3 are heavily unit-tested; E4 adds a small number of **API-level integration tests** to verify contracts between layers (DB → E2 → E3 → endpoints).

E4-Design

1. DTOs for Sessions

ThesisEvaluationSessionResult

```
class ThesisEvaluationSessionResult(BaseModel):
    thesis_id: str
    evaluation: ThesisEvaluationResult
    review: ThesisReview
    trade_plan: TradePlan | None = None
    evaluated_at: datetime
```

- review is **required** – if we can't produce one, the session fails and no result is returned.
- trade_plan is optional and purely advisory for E5.

Alert

```
class Alert(BaseModel):
    thesis_id: str
    thesis_title: str
    message: str
    observation_id: str | None = None
    timestamp: datetime
```

- thesis_title included to avoid extra DB lookups in UI.

DailyContext (LLM input)

```
class DailyContext(BaseModel):
    date: date
    portfolio_value: float | None
    portfolio_change_pct: float | None
    risk_snapshot: dict[str, Any]  # e.g. from get_portfolio_depth()
    alerts: list[str]               # human-readable alert messages
    notable_observations: list[str] # short texts/snippets
```

DailyUpdateSessionResult

```
class DailyUpdateSessionResult(BaseModel):
    date: date
    portfolio_snapshot: PortfolioSnapshot
    alerts: list[Alert]
    summary: DailySummary
```

Optional cross-thesis:

```
class CrossThesisSessionResult(BaseModel):
    thesis_ids: list[str]
    report: CrossThesisReport
```

---

2. Persistence Interfaces

We **make persistence explicit**; no "magic logging."

```
class EvaluationRepository:
    def upsert_thesis_evaluation(
        self,
        thesis_id: str,
        evaluation: ThesisEvaluationResult,
        review: ThesisReview,
        evaluated_at: datetime,
    ) -> None:
        """Either insert new or overwrite the latest evaluation for this
thesis."""
```

- Implementation: one row per thesis (or one "latest" row per thesis), e.g. JSONB columns; overwrite semantics avoid unbounded history. If you want history later, add a version/timestamp key, but for eval harness "latest only" is fine.

```
class AlertRepository:
    def insert_many(self, alerts: list[Alert]) -> None: ...
    def list_for_date(self, date: date) -> list[Alert]: ...
class DailySummaryRepository:
    def upsert_summary(self, date: date, summary: DailySummary) -> None: ...
    def get_summary(self, date: date) -> DailySummary | None: ...
```

DataAccess is extended to expose these repos (or wrap them):

```
class DataAccess:
    thesis_repo: ThesisRepository
    trade_repo: TradeRepository
    price_source: PriceSource
    evaluation_repo: EvaluationRepository
    alert_repo: AlertRepository
    daily_summary_repo: DailySummaryRepository
    # plus get_current_portfolio(), get_portfolio_depth(),
get_recent_observations()...
```

3. Session Classes

3.1 ThesisEvaluationSession

```
class ThesisEvaluationSession:
    def __init__(
        self,
        data_access: DataAccess,
        eval_service: ThesisEvaluationService,
        llm_tools: LLMTools,
        exec_adapter: PaperExecutionAdapter | None = None,
    ) -> None:
        self.data = data_access
        self.eval_service = eval_service
        self.llm = llm_tools
        self.exec_adapter = exec_adapter

    def run(self, thesis_id: str) -> ThesisEvaluationSessionResult:
        # 1. Load thesis
        thesis = self.data.thesis_repo.get_by_id(thesis_id)
        if thesis is None:
            raise ThesisNotFoundError(thesis_id)

        # 2. Quant evaluation
        eval_result = self.eval_service.evaluate_thesis(thesis)

        # 3. LLM review (hard requirement)
        review = self.llm.review_thesis(thesis, eval_result)  # may raise

        # 4. Optional trade plan (non-fatal if it fails)
        trade_plan: TradePlan | None = None
        if self.exec_adapter is not None:
            try:
                # For evaluation phase we can use a fixed notional or
configuration value
                trade_plan = self.exec_adapter.create_plan_from_thesis(
                    thesis=thesis,
                    total_value=1_000_000.0,
                )
            except Exception:
                # Log and move on - evaluation does not depend on plan
success
                trade_plan = None

        # 5. Atomic persistence
        evaluated_at = datetime.utcnow()
        self.data.evaluation_repo.upsert_thesis_evaluation(
            thesis_id=thesis_id,
            evaluation=eval_result,
            review=review,
            evaluated_at=evaluated_at,
        )

        # 6. Assemble result DTO
```

```
        return ThesisEvaluationSessionResult(
            thesis_id=thesis_id,
            evaluation=eval_result,
            review=review,
            trade_plan=trade_plan,
            evaluated_at=evaluated_at,
        )
```

Error semantics:

- ThesisNotFoundError → 404 at API layer.
- Any evaluate_thesis error (e.g. missing price data) → 400 (invalid thesis / insufficient data).
- Any LLM error → 500, no DB writes, caller sees failure and can retry.

3.2 DailyUpdateSession

```
class DailyUpdateSession:
    def __init__(self, data_access: DataAccess, llm_tools: LLMTools) -> None:
        self.data = data_access
        self.llm = llm_tools

    def run(self, as_of: date | None = None) -> DailyUpdateSessionResult:
        run_date = as_of or date.today()

        # 1. Active theses and portfolio
        theses = self.data.thesis_repo.list_active()
        portfolio = self.data.get_current_portfolio()
        depth = self.data.get_portfolio_depth()

        # 2. Observations + Alerts
        recent_obs = self.data.get_recent_observations(limit=50)  #
implementation-specific
        alerts: list[Alert] = []

        for thesis in theses:
            related_obs = [o for o in recent_obs if thesis.id in getattr(o,
"thesis_refs", [])]
            for obs in related_obs:
                if getattr(obs, "actionable", False) and obs.disconfirmer:  #
or any explicit flag
                    alerts.append(
                        Alert(
                            thesis_id=thesis.id,
                            thesis_title=thesis.title,
                            message=obs.disconfirmer_message or
obs.text[:200],
                            observation_id=obs.id,
                            timestamp=obs.timestamp,
                        )
                    )
```

```
        # Persist alerts even if LLM dies
        if alerts:
            self.data.alert_repo.insert_many(alerts)

        # 3. Build DailyContext
        prev_snapshot = self.data.daily_summary_repo.get_summary(run_date -
timedelta(days=1))
        prev_value = getattr(prev_snapshot, "portfolio_value", None) if
prev_snapshot else None

        portfolio_value = portfolio.totals.portfolio_value if portfolio is
not None else None
        portfolio_change_pct: float | None = None
        if portfolio_value is not None and prev_value:
            portfolio_change_pct = (portfolio_value / prev_value - 1.0) *
100.0

        context = DailyContext(
            date=run_date,
            portfolio_value=portfolio_value,
            portfolio_change_pct=portfolio_change_pct,
            risk_snapshot=depth.to_dict() if hasattr(depth, "to_dict") else
{},
            alerts=[f"{a.thesis_title}: {a.message}" for a in alerts],
            notable_observations=[obs.text[:280] for obs in recent_obs[:3]],
        )

        # 4. LLM summary (soft requirement)
        try:
            summary = self.llm.daily_summary(context)
        except Exception:
            summary = DailySummary(
                key_narratives=[],
                risk_highlights=[],
                thesis_references=[],
                insufficient_context=True,
            )

        # 5. Persist summary
        self.data.daily_summary_repo.upsert_summary(run_date, summary)

        # 6. Result DTO
        return DailyUpdateSessionResult(
            date=run_date,
            portfolio_snapshot=portfolio,
            alerts=alerts,
            summary=summary,
        )
```

Key design choice: **alerts and portfolio are not blocked by LLM failure**. LLM failure only flips summary.insufficient_context.

3.3 Optional: CrossThesis / Intuition

These are thin wrappers over E3 tools; nothing complex:

```
class CrossThesisSession:
    def __init__(self, data_access: DataAccess, llm_tools: LLMTools) -> None:
        self.data = data_access
        self.llm = llm_tools

    def run(self, thesis_ids: list[str]) -> CrossThesisSessionResult:
        theses = [self.data.thesis_repo.get_by_id(tid) for tid in thesis_ids]
        if any(t is None for t in theses):
            raise ValueError("Unknown thesis id in list")
        report = self.llm.cross_theses(theses)  # may raise
        return CrossThesisSessionResult(thesis_ids=thesis_ids, report=report)
```

Intuition Q&A is 1:1 with llm_query_intuition.

---

4. API Endpoints (FastAPI)

Assuming dependency providers get_data_access, get_eval_service, get_llm_tools, get_exec_adapter.

```
router = APIRouter(prefix="/api/v1")

@router.post("/thesis/{thesis_id}/evaluate",
response_model=ThesisEvaluationSessionResult)
def evaluate_thesis(
    thesis_id: str,
    data_access: DataAccess = Depends(get_data_access),
    eval_service: ThesisEvaluationService = Depends(get_eval_service),
    llm_tools: LLMTools = Depends(get_llm_tools),
    exec_adapter: PaperExecutionAdapter = Depends(get_exec_adapter),
):
    session = ThesisEvaluationSession(data_access, eval_service, llm_tools,
exec_adapter)
    try:
        return session.run(thesis_id)
    except ThesisNotFoundError as e:
        raise HTTPException(status_code=404, detail=str(e))
    except EvaluationInputError as e:
        raise HTTPException(status_code=400, detail=str(e))
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Thesis evaluation
failed: {e}")
@router.post("/session/daily-update",
response_model=DailyUpdateSessionResult)
def daily_update(
    data_access: DataAccess = Depends(get_data_access),
    llm_tools: LLMTools = Depends(get_llm_tools),
):
    session = DailyUpdateSession(data_access, llm_tools)
    try:
        return session.run()
```

```
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Daily update failed:
{e}")
```

Optional:

- POST /thesis/compare → wraps CrossThesisSession.
- POST /intel/qa → wraps llm_query_intuition with observation retrieval and Pydantic request/response models.

E4-Testing Plan

Tests are **API-level integration** using TestClient + dependency overrides, with **LLM and price data stubbed**.

1. Thesis Evaluation Flow

Setup:

1. Use test DB / in-memory DB.
2. Insert a simple Thesis:
    - One leg: asset "TEST", direction LONG, size 1.0.
3. Configure a TestPriceSource used by ThesisEvaluationService that returns deterministic prices (e.g. 100 → 110).
4. Monkeypatch LLMTools.review_thesis to return a fixed ThesisReview.

Test:

- Call POST /api/v1/thesis/{id}/evaluate.
- Assert:
    - HTTP 200.
    - response.json() contains:
        - thesis_id == id.
        - evaluation.total_return ~ 0.10 (or whatever test data implies).
        - review.critique == "Looks good" (or whatever dummy).
        - trade_plan is either null (if adapter disabled) or matches stubbed plan.
    - DB: evaluation_repo.upsert_thesis_evaluation was called once and row exists.

Error tests:

- Unknown thesis ID → 404.
- Monkeypatch evaluate_thesis to raise EvaluationInputError → 400.
- Monkeypatch LLMTools.review_thesis to raise → 500 and **no new evaluation record** in DB.

2. Daily Update Flow

Setup:

1. Insert one **ACTIVE** thesis with a disconfirmer tied to a specific pattern or explicitly via an Observation flag.
2. Insert a trade for that thesis in TradeRepository.
3. Configure price source for up-to-date valuation.
4. Insert observations:
    - At least one Observation with thesis_refs=[thesis_id], actionable=True, and disconfirmer=True (or equivalent).
5. Monkeypatch LLMTools.daily_summary to return a deterministic DailySummary.

Test success path:

- Call POST /api/v1/session/daily-update.
- Assert:
    -
    - alerts list includes at least one alert with the correct thesis_id.
    - summary.key_narratives == stubbed content.
    - summary.insufficient_context == False.
    - alert_repo.insert_many was called and row(s) exist.
    - daily_summary_repo.upsert_summary was called and row exists.

Test LLM failure degradation:

- Monkeypatch LLMTools.daily_summary to raise RuntimeError.
- Call endpoint.
- Assert:
    -
    - Alerts still present and persisted.
    - summary.insufficient_context == True.
    - summary.key_narratives and summary.risk_highlights empty.

- No crash.

Test “empty portfolio”:

- No active theses, no trades.
- Call daily-update.
- Assert:
  -
  - alerts == [].
  - Summary either has insufficient_context=True or a generic stub (depending on how stub is configured), but nothing raises.

3. Optional: CrossThesis / QA

- Cross-thesis: two dummy theses, LLM stub returns a fixed CrossThesisReport. Endpoint returns 200, and JSON matches stub.
- QA: stub llm_query_intuition and verify endpoint passes question/obs correctly and returns stubbed IntuitionAnswer.

E4 – Risks & Dependencies (Tight)

- **Schema drift**: Any change in ThesisEvaluationResult, DailySummary, etc. must be reflected in session DTOs and tests. Otherwise, integration tests will break – which is desirable.
- **Alert logic is intentionally simple** (actionable + thesis_refs/disconfirmer). It will miss subtle cases; that’s acceptable for evaluation, but do not pretend it’s “production-grade monitoring.”
- **State across days**: Using yesterday’s portfolio_value from DailySummaryRepository is a hack. For a real environment, you’d store explicit daily portfolio snapshots; here it’s enough to show wiring.
- **LLM coupling**: Thesis evaluation hard-depends on LLM. If you later want “quant-only” evaluation, you’d either:
  - add a skip_llm flag, or
  - split “QuantEvaluationSession” from “FullEvaluationSession”.
- **Concurrency**: Two simultaneous evaluations of the same thesis will race on upsert_thesis_evaluation. Last-writer-wins is acceptable for this harness.

This E4 spec is now concrete: clear DTOs, deterministic error semantics, explicit persistence, and a minimal but real daily loop that can be hit from a UI and tested end-to-end.